Assignment 5 - Strategy Pattern

Option A - Navigation / Route Planning

Student: Baimagambet Gauhar

1. Introduction

The chosen domain is navigation. Given a start point and a destination, a route can be planned in different ways depending on how a person travels. The algorithm family is route planning by travel mode: CarStrategy, WalkStrategy and CycleStrategy. Each strategy computes the distance and travel time using its own rules (a detour factor for the type of path and an average speed). The Navigator (Context) holds the current strategy and delegates planning to it, and the strategy can be replaced at runtime.

2. UML class diagram

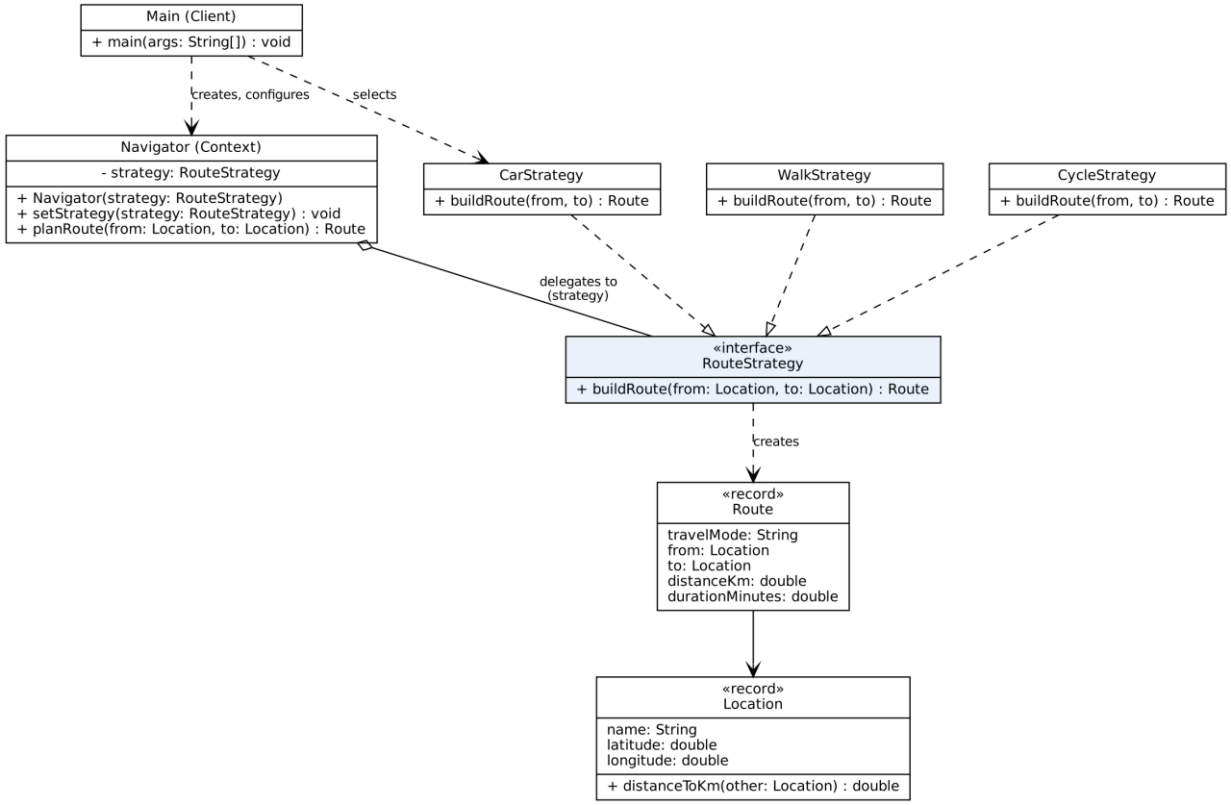


Figure 1. Context (Navigator), Strategy interface (RouteStrategy), Concrete Strategies (Car, Walk, Cycle) and Client (Main).

3. Clean Code principles applied

3.1 Elimination of conditional logic in favor of polymorphism

Before (typical approach):

```
public Route planRoute(Location from, Location to, String mode) {
    if (mode.equals("car")) {
        ...
    } else if (mode.equals("walk")) {
        ...
    } else if (mode.equals("cycle")) {
        ...
    }
}
```

After:

```
public Route planRoute(Location from, Location to) {
    return strategy.buildRoute(from, to);
}
```

The algorithm is chosen by which object is plugged in, not by if/switch.

3.2 Meaningful, intention-revealing names

```
private static final double ROAD_DETOUR_FACTOR = 1.30;
private static final double AVERAGE_SPEED_KMH = 50.0;
```

Names such as ROAD_DETOUR_FACTOR, AVERAGE_SPEED_KMH, distanceToKm and buildRoute state the meaning and unit, so no comments are needed to explain magic numbers.

3.3 Single Responsibility

```
public class WalkStrategy implements RouteStrategy {
    @Override
    public Route buildRoute(Location from, Location to) { ... }
}
```

Each strategy has exactly one reason to change: its own algorithm. Location only knows geometry, Route only carries results, and Navigator only delegates.

3.4 Context remains algorithm-agnostic

```
public class Navigator {
    private RouteStrategy strategy;   // depends only on the interface
    ...
}
```

Navigator imports RouteStrategy, never CarStrategy, WalkStrategy or CycleStrategy. It has no knowledge of how routes are computed.

3.5 Easy extensibility (Open/Closed)

```
RouteStrategy scooter = (from, to) -> {
    double distanceKm = from.distanceToKm(to) * 1.15;
    return new Route("E-scooter", from, to, distanceKm, distanceKm / 20.0 * 60);
};
navigator.setStrategy(scooter);
```

A new travel mode is added without editing Navigator or any existing strategy.

3.6 Immutability and fail-fast validation

```
public record Location(String name, double latitude, double longitude) { ... }

this.strategy = Objects.requireNonNull(strategy, "strategy must not be null");
```

Location and Route are immutable records, and Navigator rejects a null strategy immediately instead of failing later with a confusing error.

4. Strategy vs. State

Route planning calls for Strategy rather than State because the algorithm is chosen from outside by the client, based on what the user wants (how they want to travel), and the strategies are independent of one another and unaware of each other. In State, the object's own behavior changes as its internal state changes, and the states usually know about each other and trigger transitions between themselves. Here, a CarStrategy never turns into a WalkStrategy by itself; the client decides when to switch. The strategies are interchangeable ways of doing the same job, not stages of a life cycle.

5. Conclusion

The Strategy pattern gives flexibility: a new travel mode needs one new class (or a lambda) and no changes to Navigator, and strategies can be swapped at runtime. The cost is a few extra classes (one interface and one class per algorithm) and the fact that the client has to know which strategies exist. For a family of algorithms that will grow, this is a good trade, since it replaces a growing if/switch chain with small, focused classes.

6. GitHub repository

https://github.com/gauharb/sdp-asn5